

Programmierung

–

Level

1.3

1.	Vorwort zu den Programmierübungen	3
2.	Ziel der Übung.....	3
3.	Was ist auf der ATC-Platine bereits vorbereitet?.....	3
4.	Neue Befehle/Konzepte in dieser Übung	4
5.	Schritt-für-Schritt: Schreibe den Code selbst	4
5.1.	Schritt 1 – Pins festlegen	4
5.2.	Schritt 2 – setup(): Ein- und Ausgänge konfigurieren	4
5.3.	Schritt 3 – Aufgabe A: Taster steuert LED direkt	5
5.4.	Schritt 4 – Aufgabe B: Taster als Schaltbefehl	5
5.5.	Schritt 5 – Aufgabe C: Umschaltlogik	5
5.6.	Schritt 6 – Blinkmuster starten und stoppen (millis())	6
6.	Challenge	9
6.1.	Challenge A – Blinkrhythmus.....	9
6.2.	Challenge B – Geschwindigkeit verändern und LEDs ausschalten	9
7.	Typische Fehler & schnelle Diagnose	10
8.	Referenzlösung Aufgabe A (nur wenn du festhängst)	11
9.	Referenzlösung Aufgabe B (nur wenn du festhängst)	12
10.	Referenzlösung Aufgabe C (nur wenn du festhängst)	13
11.	Referenzlösung Bonus (nur wenn du festhängst)	14
12.	Referenzlösung Challenge A (nur wenn du festhängst).....	16
13.	Referenzlösung Challenge B (nur wenn du festhängst).....	17
14.	Abschluss	19

1. Vorwort zu den Programmierübungen

Die folgenden Programmierübungen sind so aufgebaut, dass sie Schritt für Schritt in die Arbeit mit Mikrocontrollern einführen.

Alle Übungen können mit dem Hardware-Einsteigerpaket Level 1 umgesetzt werden und bauen inhaltlich aufeinander auf.

Zu Beginn werden grundlegende Konzepte vermittelt und einfache Aufgaben umgesetzt. Mit jeder weiteren Übung werden neue Funktionen ergänzt und bestehende Inhalte erweitert.

Ziel ist es, ein solides Verständnis für den Aufbau von Programmen, den Umgang mit der Arduino IDE und das Zusammenspiel von Hard- und Software zu entwickeln.

Die Übungen sind bewusst übersichtlich gehalten und sollen dazu ermutigen, eigene Anpassungen vorzunehmen und mit dem Code zu experimentieren.

Es wird empfohlen, die Programmierübungen in der vorgesehenen Reihenfolge durchzuführen, da spätere Aufgaben auf den vorherigen aufbauen.

So entsteht nach und nach ein nachvollziehbarer Lernpfad, der als Grundlage für weiterführende Projekte dient.

2. Ziel der Übung

In dieser Programmierübung lernst du, zwei Taster als digitale Eingänge zu verwenden und damit die LEDs auf der ATC-Level-1-Platine zu steuern.

Am Ende kannst du:

- Tasterzustände auslesen
- LEDs direkt über Taster schalten
- einfache Logiken und Zustände umsetzen
- (Bonus) Blinkmuster starten und stoppen, ohne dass das Programm blockiert

3. Was ist auf der ATC-Platine bereits vorbereitet?

Auf der ATC-Level-1-Platine sind folgende Bauteile angeschlossen:

- **LED 1** ist an **Pin D5**
- **LED 2** ist an **Pin D6**
- **Taster 1** ist an **Pin D7**
- **Taster 2** ist an **Pin D8**

Du musst in dieser Übung nichts verdrahten.

Die Taster sind so ausgelegt, dass sie mit der internen Pull-Up-Schaltung des Arduino verwendet werden.

4. Neue Befehle/Konzepte in dieser Übung

In dieser Übung kommen erstmals digitale Eingänge hinzu.

Neue Befehle sind:

- `digitalRead()`
- `INPUT_PULLUP`
- logische Entscheidungen mit `if / else`

Hinweis:

Bei Verwendung von `INPUT_PULLUP` gilt:

- Taster nicht gedrückt → HIGH
- Taster gedrückt → LOW

5. Schritt-für-Schritt: Schreibe den Code selbst

5.1. Schritt 1 – Pins festlegen

Erstelle einen neuen Sketch und lege folgende Pins als Konstanten oder Variablen fest:

- LED 1
- LED 2
- Taster 1
- Taster 2

Aufgabe:

Nutze die Pin-Nummern 5, 6, 7 und 8.

Selbstcheck:

Du hast jetzt vier klar benannte Pins im Code.

5.2. Schritt 2 – `setup()`: Ein- und Ausgänge konfigurieren

In `setup()` musst du:

- die beiden LED-Pins als OUTPUT
- die beiden Taster-Pins als `INPUT_PULLUP`

konfigurieren.

Selbstcheck:

Im `setup()` stehen jetzt vier `pinMode()`-Aufrufe.

5.3. Schritt 3 – Aufgabe A: Taster steuert LED direkt

Jetzt reagiert das Programm direkt auf Benutzereingaben.

Aufgabe:

- Taster 1 gedrückt → LED 1 an
- Taster 1 losgelassen → LED 1 aus
- Taster 2 gedrückt → LED 2 an
- Taster 2 losgelassen → LED 2 aus

Hinweis:

Denke daran, dass ein gedrückter Taster bei INPUT_PULLUP den Wert LOW liefert.

Selbstcheck:

Die LEDs reagieren sofort und nur solange der jeweilige Taster gedrückt wird.

5.4. Schritt 4 – Aufgabe B: Taster als Schaltbefehl

Jetzt sollen die Taster Zustände ändern, nicht nur direkt reagieren.

Aufgabe:

- Taster 1 → beide LEDs an
- Taster 2 → beide LEDs aus

Die LEDs behalten ihren Zustand auch nach dem Loslassen der Taster.

Selbstcheck:

Ein kurzer Tastendruck reicht aus, um den Zustand zu ändern.

5.5. Schritt 5 – Aufgabe C: Umschaltlogik

Jetzt wird die Logik erweitert.

Aufgabe:

- Taster 1 → LED 1 an, LED 2 aus
- Taster 2 → LED 2 an, LED 1 aus

Selbstcheck:

Es ist immer nur eine LED aktiv.

5.6. Schritt 6 – Blinkmuster starten und stoppen (millis())

Jetzt kommt eine Aufgabe, bei der delay() an seine Grenze stößt.

Ziel:

- Taster 1 startet ein abwechselndes Blinkmuster
- Taster 2 beendet das Blinken
- Die Taster müssen jederzeit reagieren

Wichtig:

Mit delay() reagieren die Taster verzögert oder gar nicht.

In dieser Aufgabe wird erstmals eine nicht blockierende Zeitsteuerung verwendet. Statt das Programm mit delay() anzuhalten, wird die vergangene Zeit mit millis() abgefragt und darauf reagiert.

Grundidee von millis()

millis() liefert die Anzahl der Millisekunden, die seit dem Start des Programms vergangen sind.

Wichtig:

- millis() **wartet nicht**
- das Programm läuft ständig weiter
- Entscheidungen werden anhand von Zeitvergleichen getroffen

Das bedeutet:

Das Programm fragt nicht: „*Warte 500 ms*“, sondern: „*Sind 500 ms vergangen?*“

Prinzip

1. Die aktuelle Zeit wird gespeichert
2. In jedem Durchlauf von loop() wird geprüft:
 - Wie viel Zeit ist vergangen?
3. Ist der gewünschte Zeitraum überschritten:
 - wird der LED-Zustand geändert
 - wird die aktuelle Zeit neu gespeichert

Beispiel: Abwechselndes Blinken mit millis()

Hinweis:

Dieses Beispiel zeigt nur das Grundprinzip.
Es ist bewusst übersichtlich gehalten.

```
// Zeitsteuerung
unsigned long letzteZeit = 0;
const unsigned long intervall = 500;

// Blink-Zustand
bool ledZustand = false;

void loop() {
    unsigned long aktuelleZeit = millis();

    if (aktuelleZeit - letzteZeit >= intervall) {
        letzteZeit = aktuelleZeit;

        ledZustand = !ledZustand;

        digitalWrite(LED1_PIN, ledZustand);
        digitalWrite(LED2_PIN, !ledZustand);
    }
}
```

Erklärung zum Beispiel

- letzteZeit merkt sich, **wann zuletzt geschaltet wurde**
- intervall bestimmt die Blinkgeschwindigkeit
- millis() wird **nur gelesen**, nicht angehalten
- der LED-Zustand wird **umgeschaltet**, nicht neu berechnet

Während dieses Codes:

- laufen die LEDs weiter
- können Taster jederzeit abgefragt werden
- bleibt das Programm reaktionsfähig

Genau das ist mit delay() nicht möglich.

Merksatz zu millis()

millis() stoppt das Programm nicht.

Es ermöglicht, Zeit zu messen, während der Code weiterläuft.

Aufgabe (Hinweise, keine Lösung):

- Merke dir die aktuelle Zeit
- vergleiche sie mit einer gespeicherten Zeit
- schalte die LEDs nur, wenn die Zeit abgelaufen ist

Selbstcheck:

Das Blinkmuster läuft und kann jederzeit gestartet oder gestoppt werden.

6. Challenge

Diese Aufgaben bauen auf der Bonusaufgabe auf und erfordern saubere Logik. Sie sind bewusst anspruchsvoll.

6.1. Challenge A – Blinkrhythmus

Aufgabe:

- Taster 1 wählt Blinkrhythmus 1
- Taster 2 wählt Blinkrhythmus 2

Beispiel:

- Rhythmus 1 → langsames Blinken (z.B. 500 ms)
- Rhythmus 2 → schnelles Blinken (z.B. 200 ms)

Anforderung:

- Der gewählte Rhythmus bleibt aktiv, bis ein anderer ausgewählt wird
- Es darf immer nur ein Rhythmus aktiv sein

6.2. Challenge B – Geschwindigkeit verändern und LEDs ausschalten

Anfangs blinken beide LEDs mit 500ms

Aufgabe:

- Taster 1 erhöht Blinkgeschwindigkeit
- Taster 2 verringert Blinkgeschwindigkeit
- Taster 1 und Taster 2 gleichzeitig betätigt stoppt das Blinken (LEDs aus)

Hilfestellung – Blinkgeschwindigkeit verändern

Um die Blinkgeschwindigkeit zu verändern, muss ein Zahlenwert im Programm angepasst werden.

Dieser Wert bestimmt, wie viel Zeit zwischen zwei Blinkwechseln vergeht.

Statt einen festen Wert zu verwenden, kannst du diesen Wert vergrößern oder verkleinern.

Grundidee

- ein **größerer Zahlenwert** → langsames Blinken
- ein **kleinerer Zahlenwert** → schnelleres Blinken

Der Arduino kann mit Zahlen **rechnen**, genau wie ein Taschenrechner.

Beispielprinzip (ohne vollständigen Code)

Angenommen, du hast eine Variable für die Blinkzeit:

```
unsigned long blinkZeit = 500;
```

Wenn du möchtest, dass das Blinken langsamer wird, kannst du den Wert vergrößern:

```
blinkZeit = blinkZeit + 100;
```

Wenn du möchtest, dass das Blinken schneller wird, kannst du den Wert verkleinern:

```
blinkZeit = blinkZeit - 100;
```

Anwendung in der Challenge

- Taster 1 gedrückt: Blinkzeit vergrößern (langsamer)
- Taster 2 gedrückt: Blinkzeit verkleinern (schneller)

Der aktuelle Wert wird dabei gespeichert und beim nächsten Blinkvorgang automatisch verwendet.

7. Typische Fehler & schnelle Diagnose

- **LEDs reagieren gar nicht:**
Prüfe pinMode() und Pin-Nummern
- **Taster reagiert „verkehrt herum“:**
Prüfe, ob du LOW als „gedrückt“ behandelst.
- **Blinken stoppt nicht:**
Prüfe, ob dein Code blockierende delay()-Aufrufe enthält.

8. Referenzlösung Aufgabe A (nur wenn du festhängst)

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Aufgabe A (Referenzlösung)

```
const int LED1_PIN = 5; // D5
```

```
const int LED2_PIN = 6; // D6
```

```
const int BTN1_PIN = 7; // D7
```

```
const int BTN2_PIN = 8; // D8
```

```
void setup() {  
  pinMode(LED1_PIN, OUTPUT);  
  pinMode(LED2_PIN, OUTPUT);  
  pinMode(BTN1_PIN, INPUT_PULLUP);  
  pinMode(BTN2_PIN, INPUT_PULLUP);  
}
```

```
void loop() {  
  bool btn1Pressed = (digitalRead(BTN1_PIN) == LOW);  
  bool btn2Pressed = (digitalRead(BTN2_PIN) == LOW);  
  
  digitalWrite(LED1_PIN, btn1Pressed ? HIGH : LOW);  
  digitalWrite(LED2_PIN, btn2Pressed ? HIGH : LOW);  
}
```

9. Referenzlösung Aufgabe B (nur wenn du festhängst)

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Aufgabe B (Referenzlösung)

```
const int LED1_PIN = 5;  
const int LED2_PIN = 6;  
const int BTN1_PIN = 7;  
const int BTN2_PIN = 8;
```

```
bool lastBtn1 = HIGH;  
bool lastBtn2 = HIGH;
```

```
void setup() {  
  pinMode(LED1_PIN, OUTPUT);  
  pinMode(LED2_PIN, OUTPUT);  
  pinMode(BTN1_PIN, INPUT_PULLUP);  
  pinMode(BTN2_PIN, INPUT_PULLUP);  
}
```

```
void loop() {  
  bool btn1 = digitalRead(BTN1_PIN);  
  bool btn2 = digitalRead(BTN2_PIN);
```

```
  // Aktion nur beim "Drücken" (Flanke HIGH -> LOW)
```

```
  if (lastBtn1 == HIGH && btn1 == LOW) {  
    digitalWrite(LED1_PIN, HIGH);  
    digitalWrite(LED2_PIN, HIGH);  
  }
```

```
  if (lastBtn2 == HIGH && btn2 == LOW) {  
    digitalWrite(LED1_PIN, LOW);  
    digitalWrite(LED2_PIN, LOW);  
  }
```

```
  lastBtn1 = btn1;  
  lastBtn2 = btn2;  
}
```

10. Referenzlösung Aufgabe C (nur wenn du festhängst)

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Aufgabe C (Referenzlösung)

```
const int LED1_PIN = 5;
const int LED2_PIN = 6;
const int BTN1_PIN = 7;
const int BTN2_PIN = 8;

bool lastBtn1 = HIGH;
bool lastBtn2 = HIGH;

void setup() {
  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);
  pinMode(BTN1_PIN, INPUT_PULLUP);
  pinMode(BTN2_PIN, INPUT_PULLUP);
}

void loop() {
  bool btn1 = digitalRead(BTN1_PIN);
  bool btn2 = digitalRead(BTN2_PIN);

  if (lastBtn1 == HIGH && btn1 == LOW) {
    digitalWrite(LED1_PIN, HIGH);
    digitalWrite(LED2_PIN, LOW);
  }
  if (lastBtn2 == HIGH && btn2 == LOW) {
    digitalWrite(LED1_PIN, LOW);
    digitalWrite(LED2_PIN, HIGH);
  }

  lastBtn1 = btn1;
  lastBtn2 = btn2;
}
```

11. Referenzlösung Bonus (nur wenn du festhängst)

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Bonus (Referenzlösung)

```
const int LED1_PIN = 5;
const int LED2_PIN = 6;
const int BTN1_PIN = 7; // Start
const int BTN2_PIN = 8; // Stop

bool lastBtn1 = HIGH;
bool lastBtn2 = HIGH;

bool blinking = false;
bool phase = false;

unsigned long lastToggle = 0;
const unsigned long interval = 500;

void setup() {
  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);
  pinMode(BTN1_PIN, INPUT_PULLUP);
  pinMode(BTN2_PIN, INPUT_PULLUP);

  digitalWrite(LED1_PIN, LOW);
  digitalWrite(LED2_PIN, LOW);
}

void loop() {
  bool btn1 = digitalRead(BTN1_PIN);
  bool btn2 = digitalRead(BTN2_PIN);

  // Start/Stop nur bei Tastendruck-Flanke
  if (lastBtn1 == HIGH && btn1 == LOW) blinking = true;
  if (lastBtn2 == HIGH && btn2 == LOW) {
    blinking = false;
    digitalWrite(LED1_PIN, LOW);
    digitalWrite(LED2_PIN, LOW);
  }

  lastBtn1 = btn1;
  lastBtn2 = btn2;

  // Nicht blockierend blinken
  if (blinking) {
    unsigned long now = millis();
```



```
if (now - lastToggle >= interval) {  
    lastToggle = now;  
    phase = !phase;  
    digitalWrite(LED1_PIN, phase ? HIGH : LOW);  
    digitalWrite(LED2_PIN, phase ? LOW : HIGH);  
}  
}  
}
```

12. Referenzlösung Challenge A (nur wenn du festhängst)

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Challenge A (Referenzlösung)

```
const int LED1_PIN = 5;
const int LED2_PIN = 6;
const int BTN1_PIN = 7; // 500 ms
const int BTN2_PIN = 8; // 200 ms

bool lastBtn1 = HIGH;
bool lastBtn2 = HIGH;

unsigned long intervalMs = 500; // Startwert
unsigned long lastToggle = 0;
bool phase = false;

void setup() {
  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);
  pinMode(BTN1_PIN, INPUT_PULLUP);
  pinMode(BTN2_PIN, INPUT_PULLUP);
}

void loop() {
  bool btn1 = digitalRead(BTN1_PIN);
  bool btn2 = digitalRead(BTN2_PIN);

  // Rhythmus auswählen (nur bei Tastendruck)
  if (lastBtn1 == HIGH && btn1 == LOW) {
    intervalMs = 500;
  }
  if (lastBtn2 == HIGH && btn2 == LOW) {
    intervalMs = 200;
  }

  lastBtn1 = btn1;
  lastBtn2 = btn2;

  // Nicht blockierendes Blinken
  unsigned long now = millis();
  if (now - lastToggle >= intervalMs) {
    lastToggle = now;
    phase = !phase;

    digitalWrite(LED1_PIN, phase ? HIGH : LOW);
    digitalWrite(LED2_PIN, phase ? LOW : HIGH);
  }
}
```


13. Referenzlösung Challenge B (nur wenn du festhängst)

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Challenge B (Referenzlösung)

```
const int LED1_PIN = 5;
const int LED2_PIN = 6;
const int BTN1_PIN = 7; // schneller
const int BTN2_PIN = 8; // langsamer

bool lastBtn1 = HIGH;
bool lastBtn2 = HIGH;

bool running = true;           // Start: Blinken läuft sofort
unsigned long intervalMs = 500; // Start: 500 ms
const unsigned long STEP = 100; // Änderung pro Tastendruck
const unsigned long MIN_INTERVAL = 100;
const unsigned long MAX_INTERVAL = 1500;

unsigned long lastToggle = 0;
bool phase = false;

void applyOutputs() {
    if (!running) {
        digitalWrite(LED1_PIN, LOW);
        digitalWrite(LED2_PIN, LOW);
        return;
    }

    // Blinkmuster (hier: abwechselnd)
    digitalWrite(LED1_PIN, phase ? HIGH : LOW);
    digitalWrite(LED2_PIN, phase ? LOW : HIGH);
}

void setup() {
    pinMode(LED1_PIN, OUTPUT);
    pinMode(LED2_PIN, OUTPUT);
    pinMode(BTN1_PIN, INPUT_PULLUP);
    pinMode(BTN2_PIN, INPUT_PULLUP);

    applyOutputs();
}

void loop() {
    bool btn1 = digitalRead(BTN1_PIN);
    bool btn2 = digitalRead(BTN2_PIN);
```

```

bool btn1Pressed = (btn1 == LOW);
bool btn2Pressed = (btn2 == LOW);

// Challenge C: beide gleichzeitig = AUS
if (btn1Pressed && btn2Pressed) {
    running = false;
    phase = false;
    applyOutputs();
} else {
    // Flanken erkennen
    bool btn1Edge = (lastBtn1 == HIGH && btn1 == LOW);
    bool btn2Edge = (lastBtn2 == HIGH && btn2 == LOW);

    // Wenn AUS: ein Tastendruck startet wieder
    if (!running && (btn1Edge || btn2Edge)) {
        running = true;
        lastToggle = millis();
        phase = false;
        applyOutputs();
    }

    // Challenge B: Geschwindigkeit ändern (nur wenn running)
    if (running) {
        if (btn1Edge) {
            // schneller => Intervall kleiner
            if (intervalMs > MIN_INTERVAL + STEP) intervalMs -= STEP;
            else intervalMs = MIN_INTERVAL;
        }
        if (btn2Edge) {
            // langsamer => Intervall größer
            if (intervalMs + STEP < MAX_INTERVAL) intervalMs += STEP;
            else intervalMs = MAX_INTERVAL;
        }
    }
}

lastBtn1 = btn1;
lastBtn2 = btn2;

// Nicht blockierendes Blinken
if (running) {
    unsigned long now = millis();
    if (now - lastToggle >= intervalMs) {
        lastToggle = now;
        phase = !phase;
        applyOutputs();
    }
}
}

```

14. Abschluss

Diese Programmierübung markiert den Übergang von einfachen Ausgaben zu echter Programmlogik.

Du hast gelernt, wie Eingaben, Zustände und zeitgesteuerte Abläufe zusammenspielen.

Damit bist du optimal auf komplexere Aufgaben vorbereitet.